✅ Instrukcja:

Jak stworzyć własny dodatek „OLED Display Server” do Home Assistant (HA OS)

Folder dodatku (AddOn’a)
<<**oled_test**>>

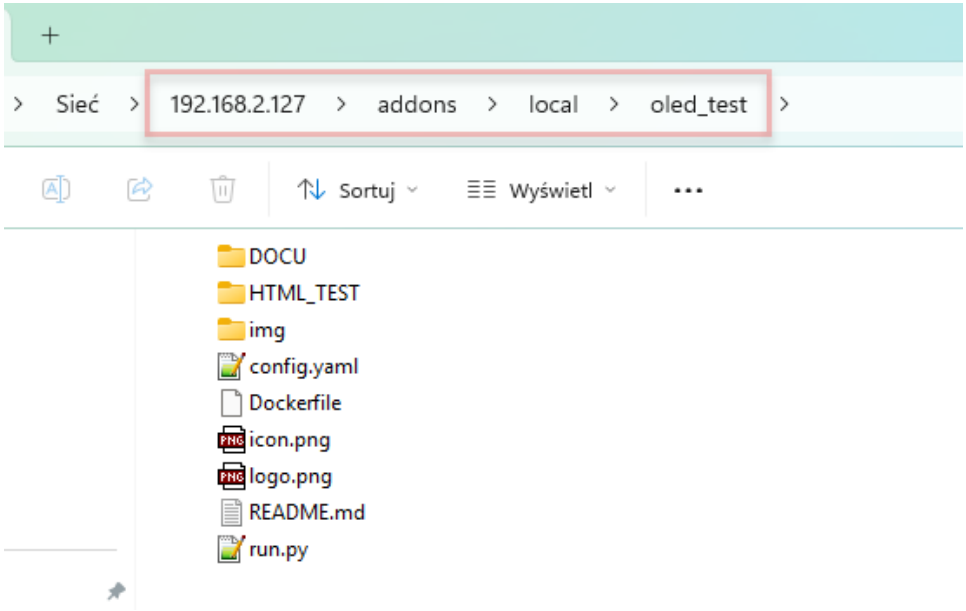
📁 1. Gdzie stworzyć foldery i pliki?

Z poziomu **komputera z Windows**, połącz się przez **Sambę** z Home Assistant:

`\\192.168.2.113\addons\local`

Utwórz tam folder:
Ścieżka końcowa:

`\\192.168.2.113\addons\local\oled_test`



W środku tego folderu utwórz pliki:

```
⚙️  config.yaml       # opis i konfiguracja dodatku (metadane dla HA)
🐬  Dockerfile        # budowa obrazu Dockera (instalacja, start)
🌐  run.py            # główny serwer Flask – logika wyświetlacza OLED
🖼️  icon.png          # ikona dodatku widoczna w Add-on Store
🖼️  logo.png          # większe logo – w panelu informacyjnym
📘  README.md         # opis dodatku (renderowany w interfejsie HA)
🖼️  img/oled_pic.png  # obrazek użyty w README.md (zrzut ekranu OLED)
```

🛠️ 2. Zawartość każdego pliku tekstowego

config.yaml

```yaml
name: OLED Display Server               # Nazwa dodatku widoczna w HA
version: "1.0"                          # Wersja dodatku
slug: oled_test          # Krótka, unikalna nazwa dodatku (używana przez HA wewnętrznie)
description: Wyswietlanie tekstu na ekranie OLED SSD1306 przez API
url: https://atnel.pl                   # Strona autora / dokumentacja
startup: services                       # Startuje z systemowymi usługami
host_network: true                      # Współdzieli sieć z hostem (dostęp
do IP)
privileged:
  - SYS_ADMIN                           # Wymaga uprawnień systemowych
devices:
  - /dev/i2c-1                          # Dostęp do magistrali I2C (OLED)
schema: {}                              # Brak parametrów konfiguracyjnych
options: {}                             # Brak opcji do ustawienia w UI
panel_icon: mdi:monitor                # Ikona panelu w HA (Material Design)
panel_title: OLED Server                # Tytuł w menu bocznym HA
arch:
  - aarch64                             # Raspberry Pi 4/5, ARM64
  - armhf                               # starsze Pi (np. 3B)
  - amd64                               # PC, NUC,  VirtualBox itp.
logo: true                              # Pokazuje logo w opisie dodatku
icon: icon.png                          # Ikona widoczna w Add-on Store
```

Dockerfile

```
#============================================================
# plik dockerfile jest plikiem sterującym procesem tworzenia dockera przez HA
# nasz addon znajdzie się właśnie w tym niezależnym dokerze/konterze (izolowany serwer) w HA
#============================================================

# Bazowy obraz Dockera (Python 3.11 w lekkiej wersji "slim")
# To punkt startowy naszego własnego Dockera tworzonego w dodatku HA
# Wszystko poniżej buduje go krok po kroku – aż do gotowego serwera OLED

FROM python:3.11-slim

#-------------------------------------------------------------

# Zainstaluj zależności systemowe i Pythonowe
# --no-install-recommends   mniej śmieci w obrazie, lżejszy Docker (~80 MB mniej)
# zlib1g-dev                pełna kompatybilność z Pillow (obsługa PNG, JPEG itp.)
# fonts-dejavu-core         zapewnia dostępność DejaVuSans.ttf dla Twojego OLED-a
# --no-cache-dir przy pip   nie trzyma śmieci z cache PIP (~100 MB mniej)
# apt-get clean + rm -rf    czyści cache APT (kolejne ~60 MB mniej)

RUN apt-get update && \
    apt-get install -y --no-install-recommends i2c-tools libfreetype6-dev
libjpeg-dev zlib1g-dev build-essential python3-dev fonts-dejavu-core && \
    pip install --no-cache-dir luma.oled pillow flask flask-cors gunicorn
&& \
    apt-get clean && rm -rf /var/lib/apt/lists/*

#-------------------------------------------------------------

# Skopiuj aplikację
# (czyli plik run.py z folderu instalacyjnego dodatku HA do budowanego obrazu Dockera)
# Ten plik trafi do wnętrza tworzonego obrazu. To nasz główny serwer Flask (API dla OLED)
COPY run.py .

#-------------------------------------------------------------

# Uruchom Gunicorna z jednym workerem i jednym wątkiem
# (czyli start naszego serwera Flask wewnątrz kontenera)
# Gunicorn to lekki serwer HTTP dla aplikacji Python
# --workers=1 i --threads=1 bo OLED przez I2C nie jest w pełni wielowątkowy
# – to bezpieczne ustawienie
# Kontener nasłuchuje na porcie 8080 (REST API do komunikacji z Home Assistantem)
CMD ["gunicorn", "--workers=1", "--threads=1", "--bind=0.0.0.0:8080",
"run:app"]
```

run.py

```
# -*- coding: utf-8 -*-

# ====================================================================
# run.py – serwer Flask do sterowania OLED SSD1306 przez REST API
# Używany w Home Assistant jako add-on (Docker + Gunicorn)
# Autor: ATNEL  |  https://atnel.pl
# ====================================================================

from flask import Flask, request, jsonify          # Flask: serwer HTTP
from flask_cors import CORS                        # CORS: zezwala na żądania z HA
(frontend)
from luma.core.interface.serial import i2c         # I2C: komunikacja z OLED
from luma.oled.device import ssd1306               # SSD1306: sterownik
wyświetlacza
from PIL import Image, ImageDraw, ImageFont        # Pillow: rysowanie grafiki i
tekstu
from threading import Lock                         # Lock: bezpieczne
współdzielenie bufora obrazu

# ====================================================================
# 1. Inicjalizacja aplikacji Flask
# ====================================================================
app = Flask(__name__)           # Tworzymy aplikację Flask
CORS(app)                       # Zezwalamy na zapytania z przeglądarki HA (CORS)

# ====================================================================
# 2. Konfiguracja OLED (SSD1306 przez I2C)
# ====================================================================
serial = i2c(port=1, address=0x3D)  # I2C: port 1, adres 0x3D (typowy dla SSD1306
to 0x3C)
device = ssd1306(serial)            # Tworzymy obiekt wyświetlacza
device.clear()                      # Czyścimy ekran
device.show()                       # Odświeżamy wyświetlacz

# ====================================================================
# 3. Wczytanie czcionek (DejaVuSans – systemowa, dostępna w Dockerze)
# ====================================================================
fonts = {}                          # Słownik: rozmiar → czcionka
for size in [10, 12, 14, 16, 18, 20]:
    fonts[str(size)] = ImageFont.truetype(
        "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", size
    )
# Czcionki są gotowe do natychmiastowego użycia – bez ponownego ładowania

# ====================================================================
# 4. Bufor obrazu (1-bitowy: czarno-biały)
# ====================================================================
image_lock = Lock()                                 # Blokada: zapobiega równoczesnemu
dostępowi do OLED
current_image = Image.new("1", device.size, "black")  # Bufor obrazu w pamięci RAM

# ====================================================================
# 5. Splash screen – wyświetlany tylko raz przy starcie
# ====================================================================
splash = Image.new("1", device.size, "black")   # Tworzymy pusty obraz
draw = ImageDraw.Draw(splash)                   # Przygotowujemy narzędzie do
rysowania
draw.text((0, 5),  "OLED SERVER", font=fonts["18"], fill="white")  # Nagłówek
draw.text((5, 32), "ready:_",     font=fonts["16"], fill="white")  # Status
device.display(splash)                          # Wyświetlamy ekran startowy

# ====================================================================
# 6. ENDPOINT: /api/oled/text – wyświetlenie tekstu
# ====================================================================
```

```python
@app.route("/api/oled/text", methods=["POST"])
def handle_oled_text():
    global current_image
    data = request.get_json(force=True)          # Odbiór danych JSON z zapytania

    # Pobieramy parametry (z domyślnymi wartościami)
    txt = data.get("txt", "").strip()            # Tekst do wyświetlenia
    x = int(data.get("x", 0))                    # Pozycja X
    y = int(data.get("y", 0))                    # Pozycja Y
    font_size = str(data.get("font", "12"))      # Rozmiar czcionki
    draw_box = bool(data.get("box", False))      # Czy narysować ramkę
    font = fonts.get(font_size, fonts["12"])     # Domyślnie czcionka 12 px

    # Rysowanie w buforze - z blokadą (bezpieczne przy wielu żądaniach)
    with image_lock:
        draw = ImageDraw.Draw(current_image)

        if draw_box:
            # Ramka wokół całego ekranu (grubość 2 px)
            draw.rectangle(
                [0, 0, device.width - 1, device.height - 1],
                outline="white", width=2
            )

        # Rysujemy tekst na ekranie
        draw.text((x, y), txt, font=font, fill="white")

        # Aktualizujemy wyświetlacz
        device.display(current_image)

    # Zwracamy odpowiedź do klienta (np. HA, Node-RED, B4A, przeglądarka)
    return jsonify({"status": "OK", "message": "Tekst wyświetlony"})

# ======================================================================
# 7. ENDPOINT: /api/oled/clear - czyszczenie ekranu
# ======================================================================
@app.route("/api/oled/clear", methods=["POST"])
def handle_clear():
    global current_image

    with image_lock:
        current_image = Image.new("1", device.size, "black")  # Nowy pusty bufor
        device.display(current_image)                         # Czyścimy fizyczny
ekran

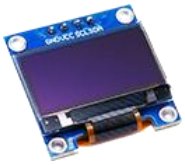
    return jsonify({"status": "OK", "message": "OLED i bufor wyczyszczony"})

# ======================================================================
# GOTOWE - serwer OLED działa:
#  - obsługuje REST API
#  - wyświetla tekst i czyści ekran
#  - uruchamiany automatycznie przez Gunicorn w Dockerze (HA add-on)
# ======================================================================
```

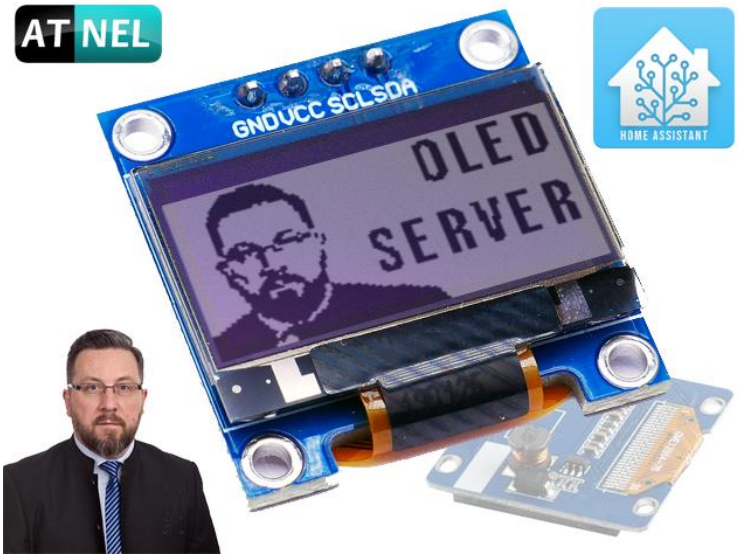
📘 README.md

OLED Display Server
To jest lokalny dodatek do Home Assistant, który pozwala wysyłać tekst na wyświetlacz OLED (SSD1306) przez sieć za pomocą REST API.
Przykłady użycia
- Wysyłaj dane z czujników na wyświetlacz OLED za pomocą dowolnych skryptów lub integracji.
- Zarządzaj tym z przeglądarki, B4A, Node-RED i in.
Screenshot
![OLED](https://atnel.pl/oled_pic.png)

🖼️ Przykładowe pliki graficzne:







🔄 3. Jak ręcznie przeładować dodatek

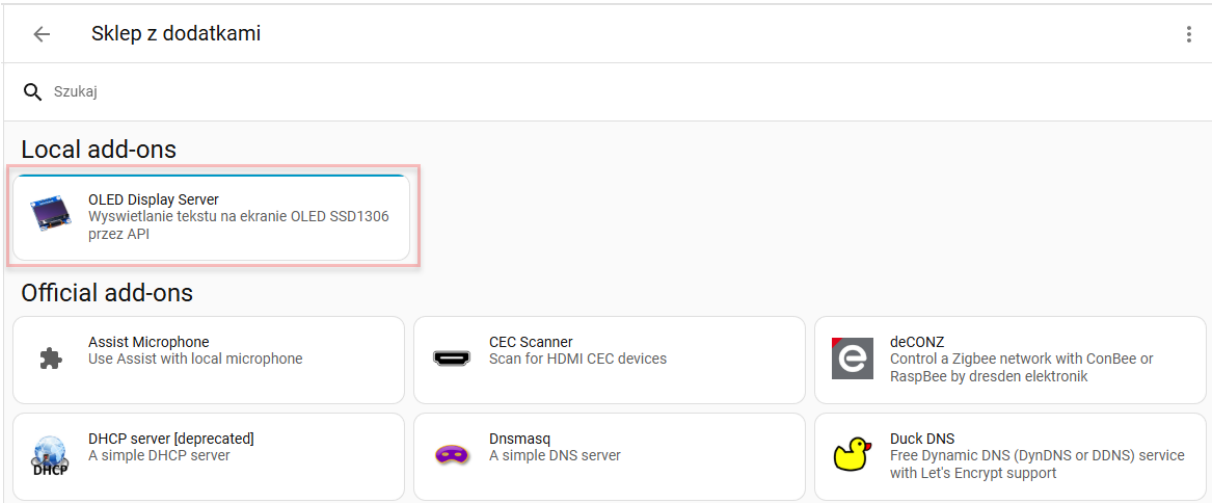
Po każdej edycji plików na Sambie:

ha supervisor reload
ha supervisor restart

Czasem może być potrzebne jeszcze:

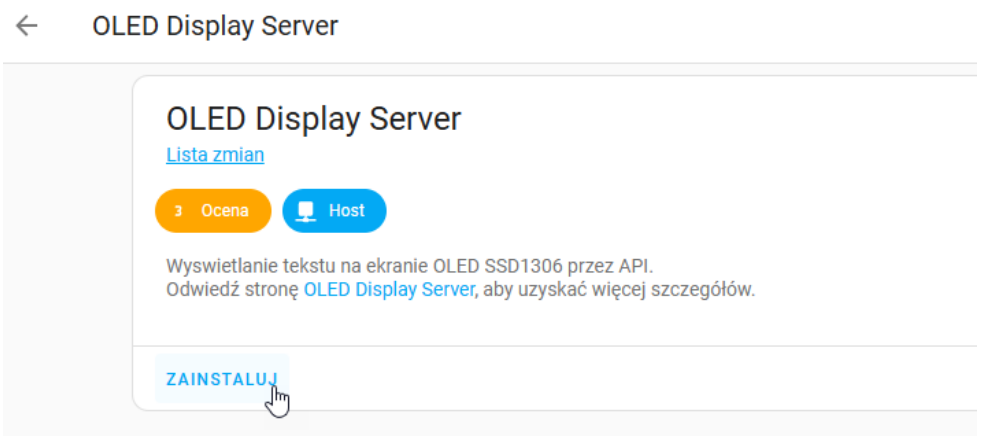
ha core restart

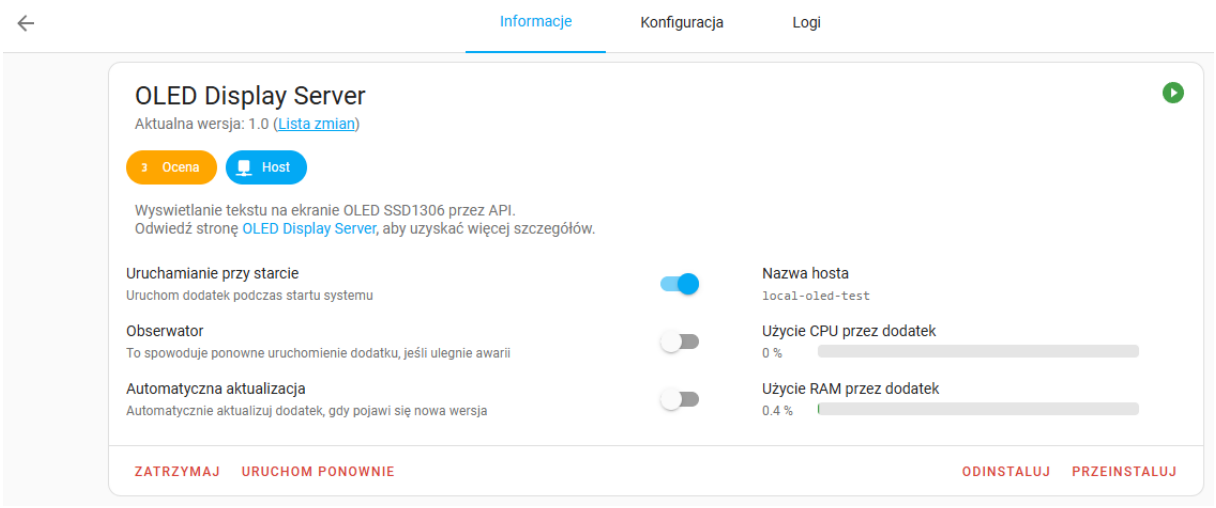
Po tym w dodatkach w sklepie powinien się pojawić nasz lokalny dodatek:



Następnie dodatek pojawi się w **Sklepie dodatków → Local add-ons** jako OLED Display Server.

Trzeba wtedy zainstalować i zachowuje się jak każdy dodatek zewnętrzny !!!!! Super





🧪 4. Jak przetestować?

Strona HTML do wysyłania tekstu na OLED

Zapisz to jako oled_test.html i otwórz w przeglądarce:

```
<!DOCTYPE html>

<html lang="pl">
<head>
  <meta charset="UTF-8">
  <title>ATNEL OLED SENDER</title>
  <style>
    body {
      margin: 0;
      font-family: 'Segoe UI', sans-serif;
      background-color: #f0f2f5;
      color: #333;
    }

    header {
```

```
  background-color: #007bff;
  color: white;
  padding: 20px 0;
  text-align: center;
  font-size: 28px;
  font-weight: bold;
  position: relative;
}

header::after {
  content: "";
  display: block;
  height: 4px;
  background: linear-gradient(to right, #007bff, transparent);
  position: absolute;
  bottom: 0;
  left: 0;
  width: 100%;
}

.container {
  max-width: 600px;
  margin: 40px auto;
  background: white;
  border-radius: 12px;
  padding: 30px;
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.1);
}

label {
  display: block;
  margin: 20px 0 5px;
  font-weight: 500;
}

label.ip-label {
  color: #007bff;
  font-weight: bold;
  margin-bottom: 8px;
}

input[type="text"],
input[type="number"],
select {
  width: 100%;
  padding: 10px;
  font-size: 16px;
  border: 1px solid #ccc;
  border-radius: 6px;
  box-sizing: border-box;
}

input[type="checkbox"] {
  margin-right: 8px;
  transform: scale(1.2);
}

.button-group {
  margin-top: 30px;
  display: flex;
  justify-content: space-between;
  gap: 10px;
}

button {
  flex: 1;
  padding: 12px;
  font-size: 16px;
  font-weight: bold;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: all 0.2s ease-in-out;
  color: white;
}

/* Główny niebieski przycisk */
```

```
    button:not(.cls-btn) {
      background: #007bff;
      box-shadow: 0 3px 8px rgba(0, 123, 255, 0.3);
    }

    button:not(.cls-btn):hover {
      background: #0056b3;
      box-shadow: 0 4px 12px rgba(0, 86, 179, 0.5);
    }

    /* Przyciemniony przycisk CLS */
    button.cls-btn {
      background: #0056b3;
      box-shadow: 0 3px 8px rgba(0, 86, 179, 0.3);
    }

    button.cls-btn:hover {
      background: #003f8c;
      box-shadow: 0 4px 12px rgba(0, 63, 140, 0.5);
    }

    #result {
      margin-top: 25px;
      font-weight: bold;
      text-align: center;
    }

    .ok { color: #28a745; }
    .error { color: #dc3545; }
  </style>
</head>
<body>

  <header>
    ATNEL OLED SENDER (POST)
  </header>

  <div class="container">
    <label class="ip-label">Adres IP serwera:</label>
    <input type="text" id="ip" value="192.168.1.139">

    <label>Tekst:</label>
    <input type="text" id="txt" value="Witaj Mirek!">

    <label>X:</label>
    <input type="number" id="x" value="10">

    <label>Y:</label>
    <input type="number" id="y" value="10">

    <label>Rozmiar czcionki:</label>
    <select id="font">
      <option value="10">10 (mała)</option>
      <option value="12" selected>12 (średnia)</option>
      <option value="14">14 (duża)</option>
      <option value="16">16 (bardzo duża)</option>
      <option value="18">18 (ogromna)</option>
      <option value="20">20 (mega)</option>
    </select>

    <label>
      <input type="checkbox" id="frame"> Rysuj ramkę
    </label>

    <div class="button-group">
      <button onclick="sendToOled()">Wyślij na OLED</button>
      <button class="cls-btn" onclick="clearOled()">CLS</button>
    </div>

    <div id="result"></div>
  </div>

  <script>
    function sendToOled() {
      const ip = document.getElementById("ip").value.trim();
      const data = {
        txt: document.getElementById("txt").value,
```

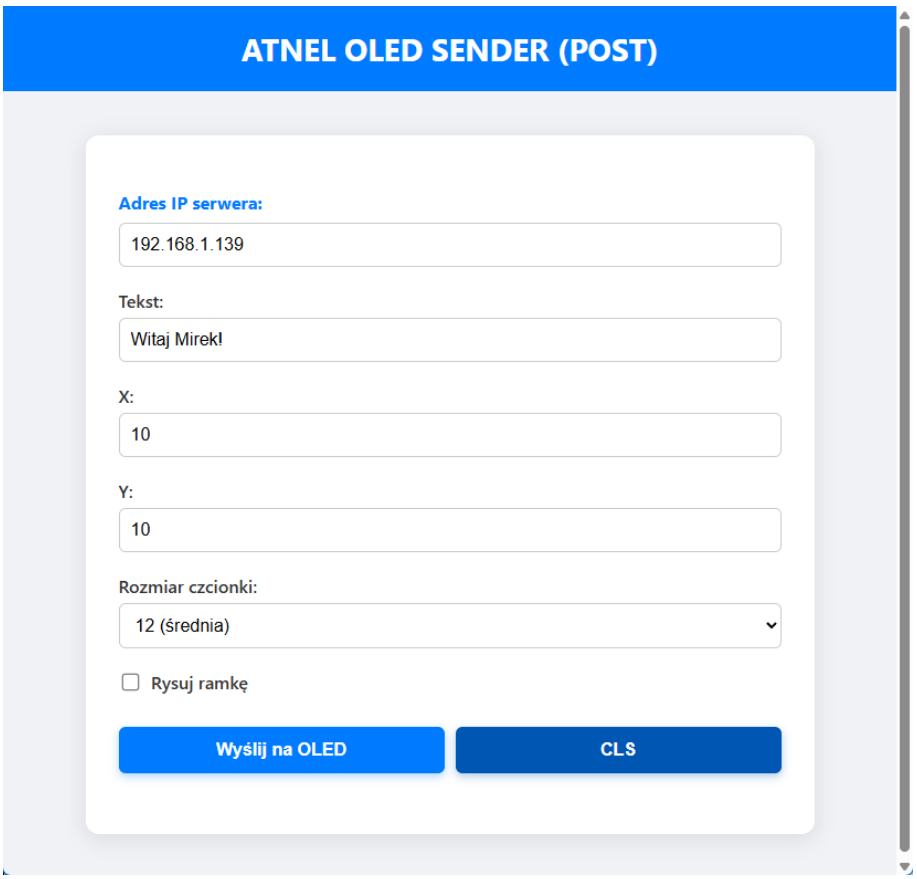
```
        x: parseInt(document.getElementById("x").value),
        y: parseInt(document.getElementById("y").value),
        font: document.getElementById("font").value,
        box: document.getElementById("frame").checked
      };

      fetch("http://" + ip + ":8080/api/oled/text", {
        method: "POST",
        headers: {
          "Content-Type": "application/json; charset=utf-8"
        },
        body: JSON.stringify(data)
      })
      .then(response => response.json())
      .then(result => {
        document.getElementById("result").innerHTML = "✔ Tekst wysłany!";
        document.getElementById("result").className = "ok";
      })
      .catch(error => {
        document.getElementById("result").innerHTML = "❌ Błąd podczas wysyłania!";
        document.getElementById("result").className = "error";
      });
    }

    function clearOled() {
      const ip = document.getElementById("ip").value.trim();

      fetch("http://" + ip + ":8080/api/oled/clear", {
        method: "POST"
      })
      .then(response => response.json())
      .then(result => {
        document.getElementById("result").innerHTML = "🧹 OLED wyczyszczony!";
        document.getElementById("result").className = "ok";
      })
      .catch(error => {
        document.getElementById("result").innerHTML = "❌ Błąd podczas czyszczenia OLED!";
        document.getElementById("result").className = "error";
      });
    }
  </script>

</body>
</html>
```



🧱 5. Jak ręcznie zbudować i uruchomić dodatek z terminala?

Kiedy dokonasz zmian np. w Dockerfile lub run.py, możesz **ręcznie przebudować** i **uruchomić** dodatek bez restartu Home Assistant.

✅ **Od teraz możesz w terminalu używać pełnych, działających komend:**

Akcja	Komenda
🔄 **Restart dodatku**	`ha addons restart local_oled_test`
▶️ **Start dodatku**	`ha addons start local_oled_test`
⏹ **Stop dodatku**	`ha addons stop local_oled_test`
🧱 **Rekompilacja dodatku**	`ha addons rebuild local_oled_test`
🔍 **Info o dodatku**	`ha addons info local_oled_test`
📄 **Logi dodatku**	`ha addons logs local_oled_test`
❌ **Odinstaluj dodatek**	`ha addons uninstall local_oled_test`

To **Supervisor** (czyli instalator dodatków w Home Assistant OS) **automatycznie dodaje prefix local_** do slugów **wszystkich dodatków z lokalnego folderu**